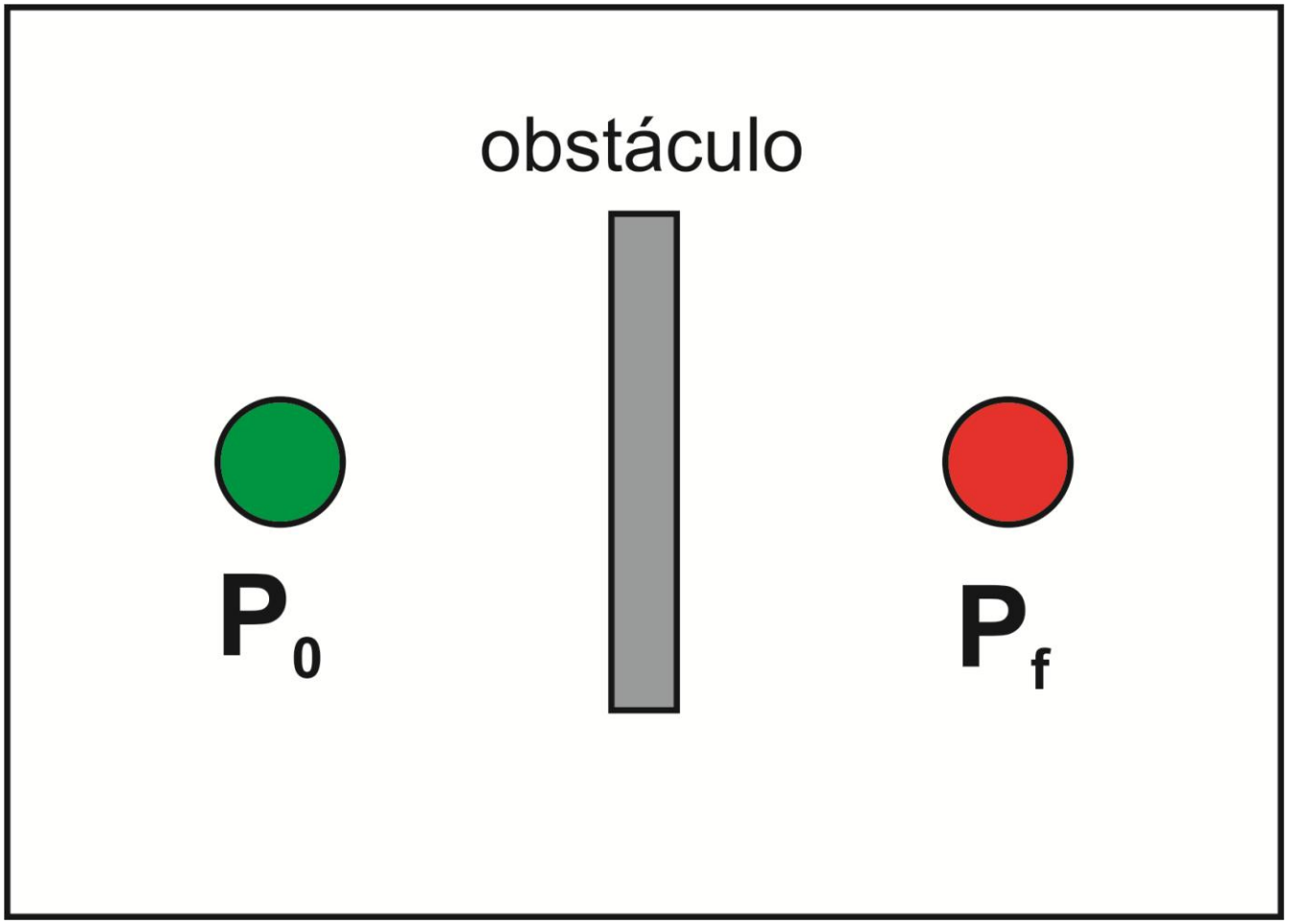
Motion Planning

Graph search: A* - practical working example

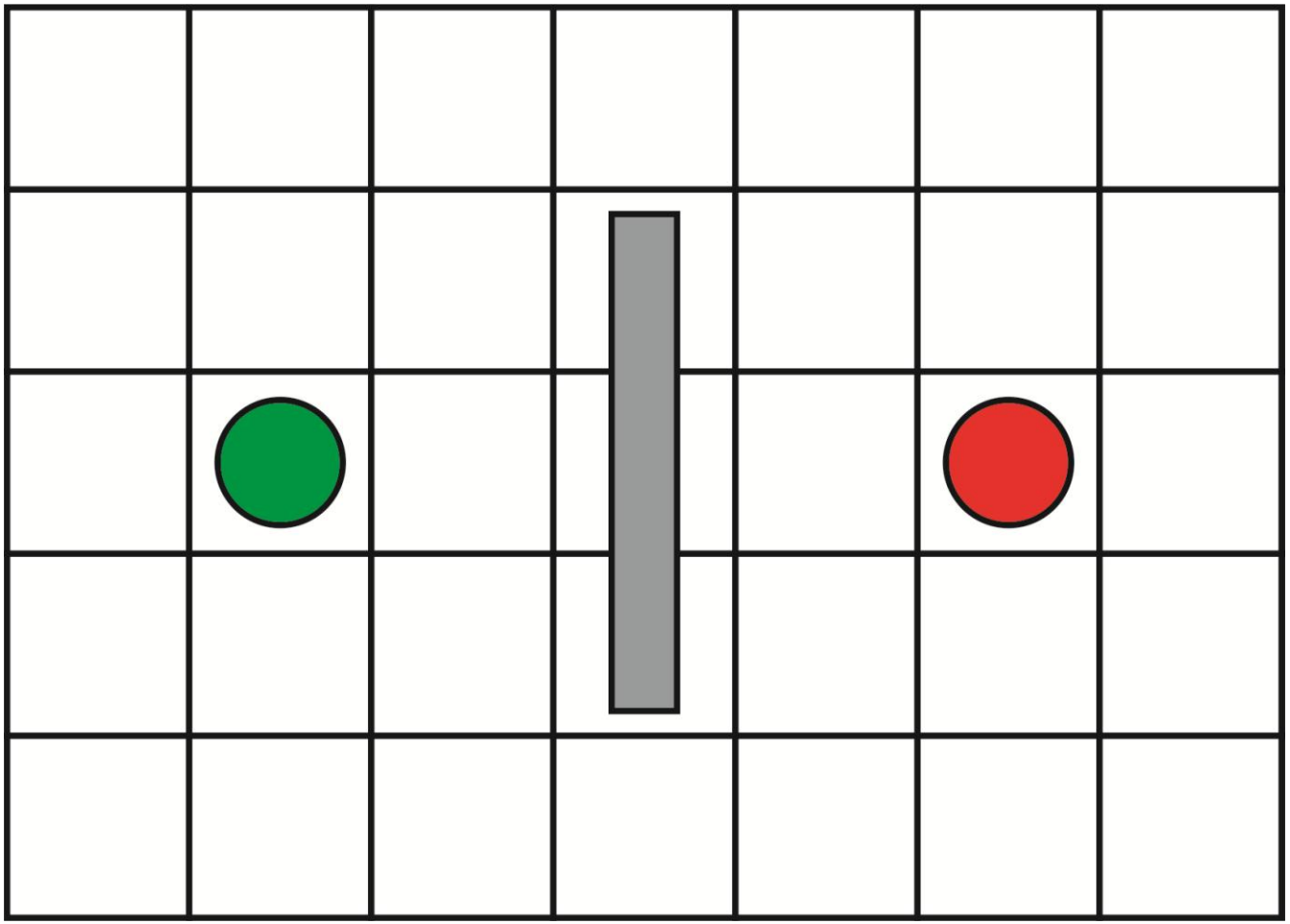
Eric Rohmer

based on material of Leonardo Olivi

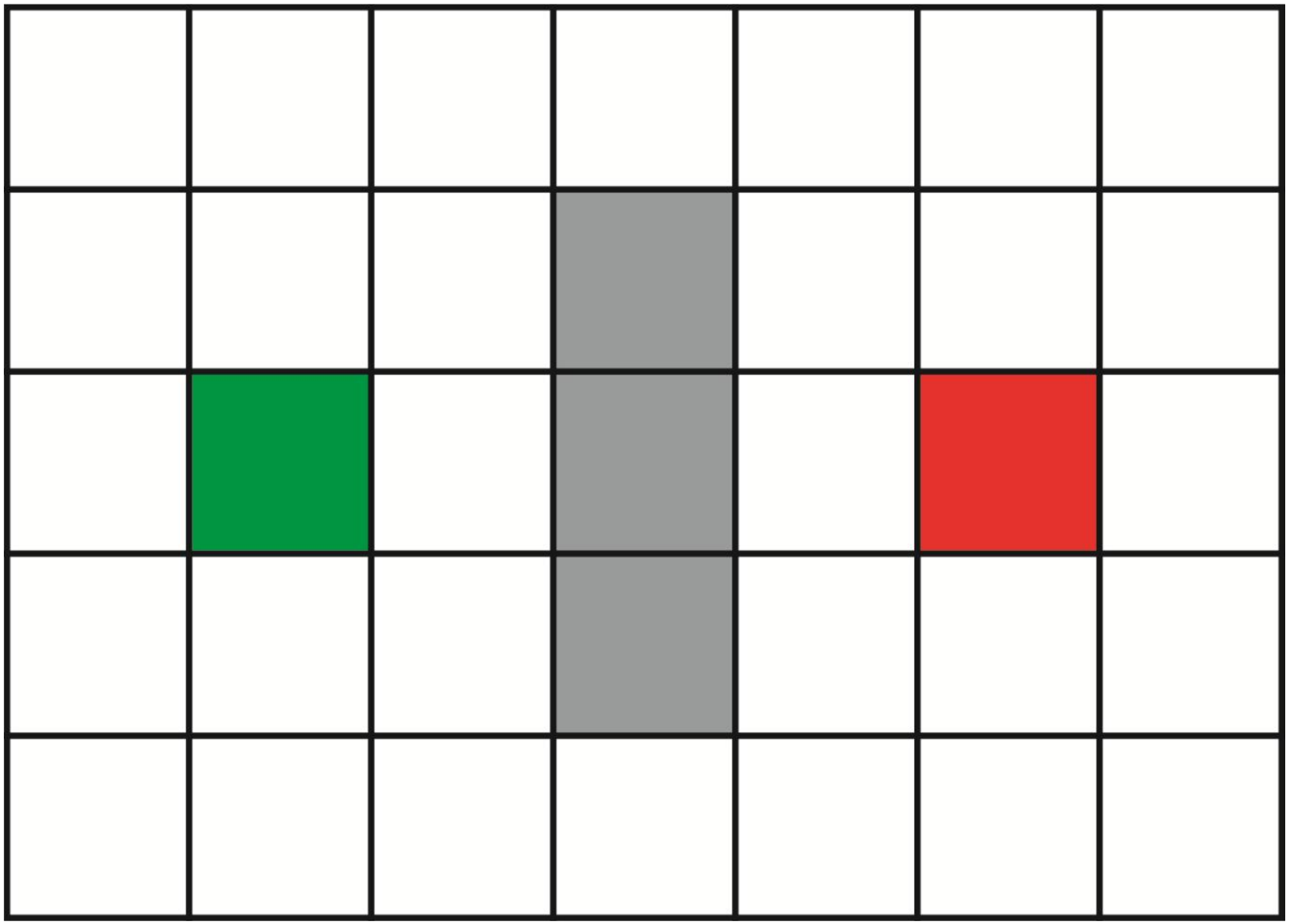
- Seja um dado mapa com uma posição inicial e uma posição objetivo com obstáculos entre elas:



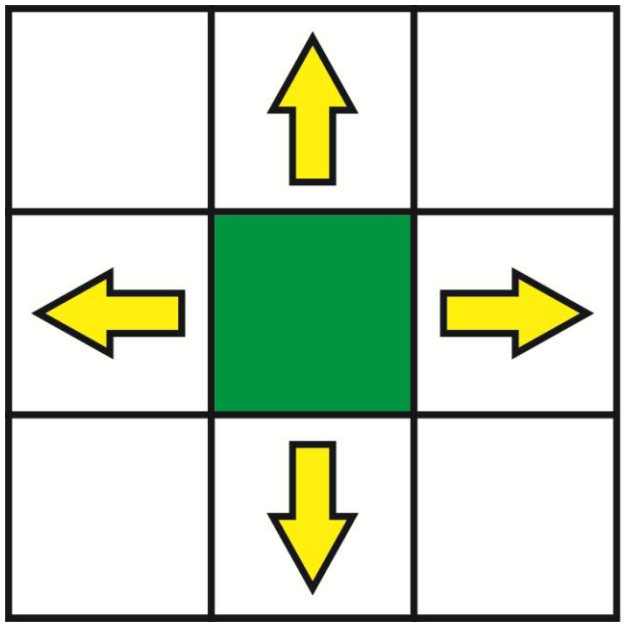
- Para não fazer a busca num espaço contínuo, discretiza-se o ambiente em células de tamanho arbitrário **L**.



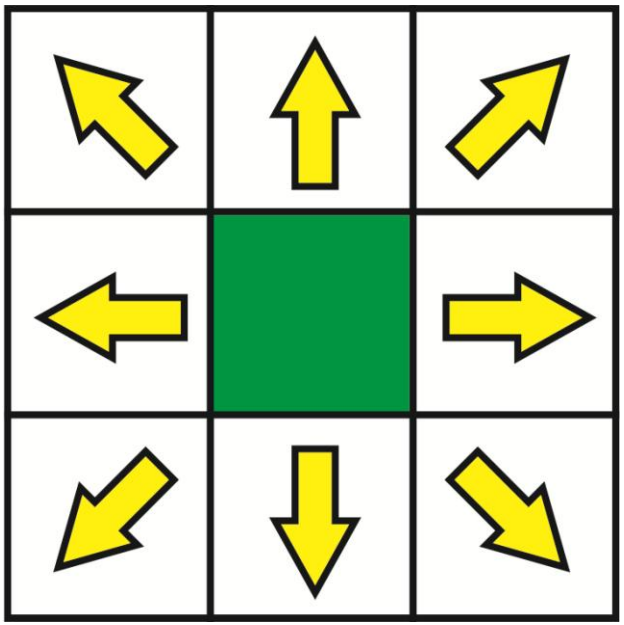
- O resultado é um mapa de células (*grid*) contendo as células inicial, final e as que contém obstáculos.



- Para pesquisar num espaço discreto, é necessário saber como as células se conectam (**conectividade** do espaço).
- As conectividades mais utilizadas são 4 e 8.



conectividade 4

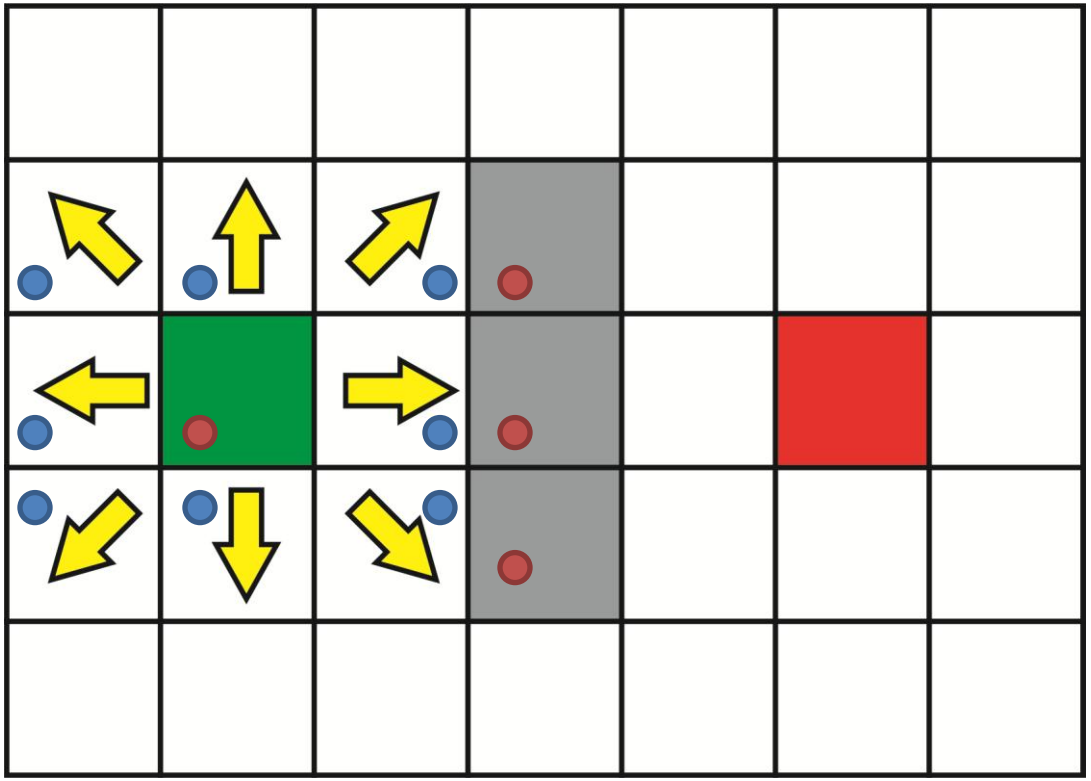


conectividade 8

- O A* fará uso de duas listas: uma lista de **células abertas** (*open list*) e uma lista de **células fechadas** (*closed list*).
- Na lista de **células abertas** constarão todas as células vizinhas das células que foram exploradas (conectividade 4 ou 8).
- Na lista de **células fechadas** constarão as células com obstáculos e as células que foram exploradas.
- Então, **utilizando conectividade 8**, expande-se a célula atual:

Lista de células abertas (inicial)

Lista de células Fechadas (inicial)



Ao explorar uma determinada célula atual A, é necessário armazenar também qual foi a célula B que levou até a célula atual A. Dessa forma, é possível avaliar se chegar na célula atual A por outros caminhos é mais vantajoso ou não.

- O algoritmo A* utiliza a seguinte função de custo:

$$F = g + h$$

- A função **g** é o custo da célula inicial até a célula atual **mais** o custo da célula atual até a célula vizinha;

$$g = d_{inicio \rightarrow atual} + d_{atual \rightarrow vizinho}$$

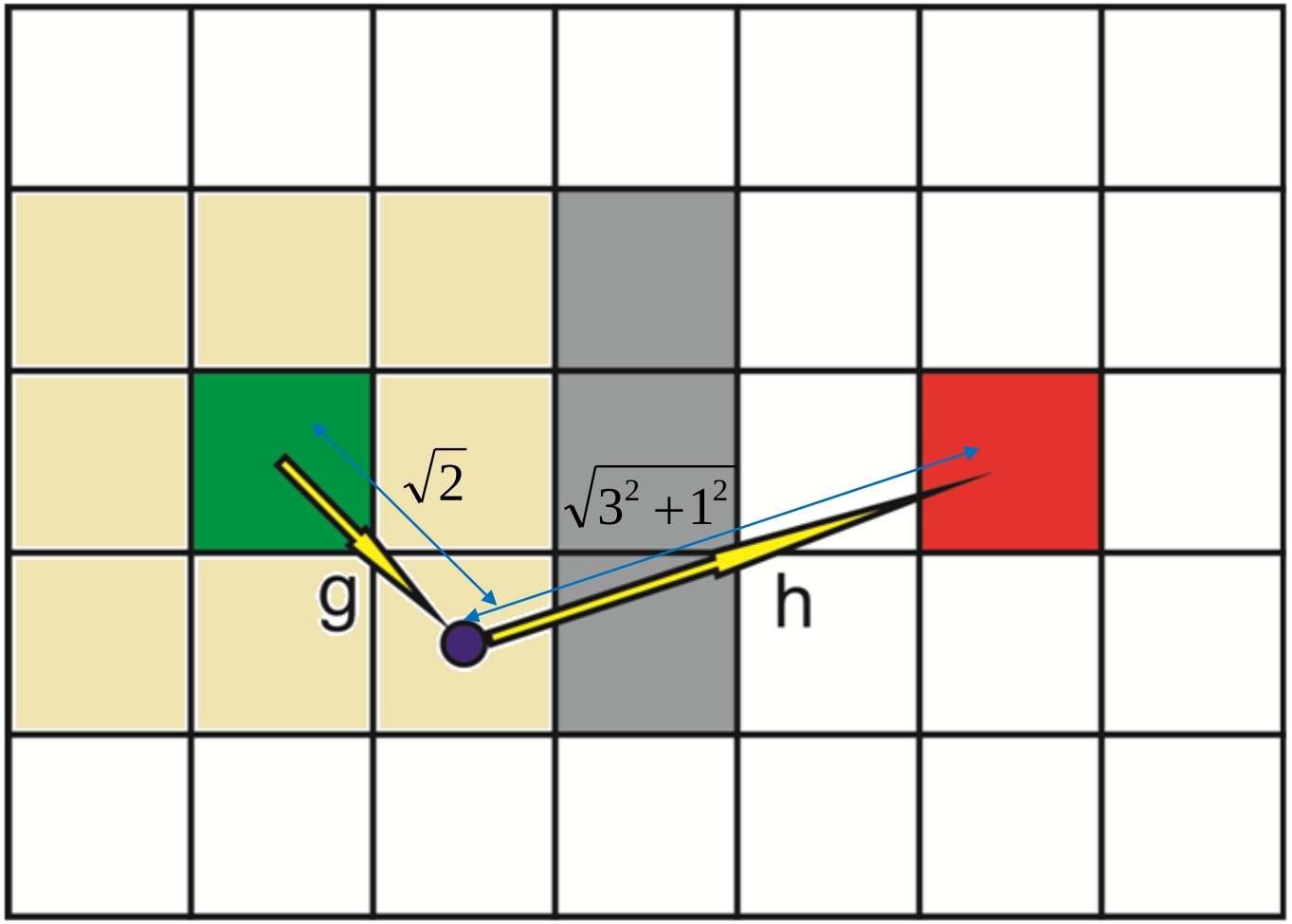
- A função **h** é e o custo da célula vizinha até a célula objetivo;

$$h = d_{vizinho \rightarrow objetivo}$$

- Uma função de custo **F** é dita otimista (admissível) quando o custo é calculado por uma função que retorna um valor menor ou igual ao valor verdadeiro (real). Por exemplo, distância Euclidiana.

$$h(x_v, y_v, x_o, y_o) = \sqrt{(x_o - x_v)^2 + (y_o - y_v)^2}$$

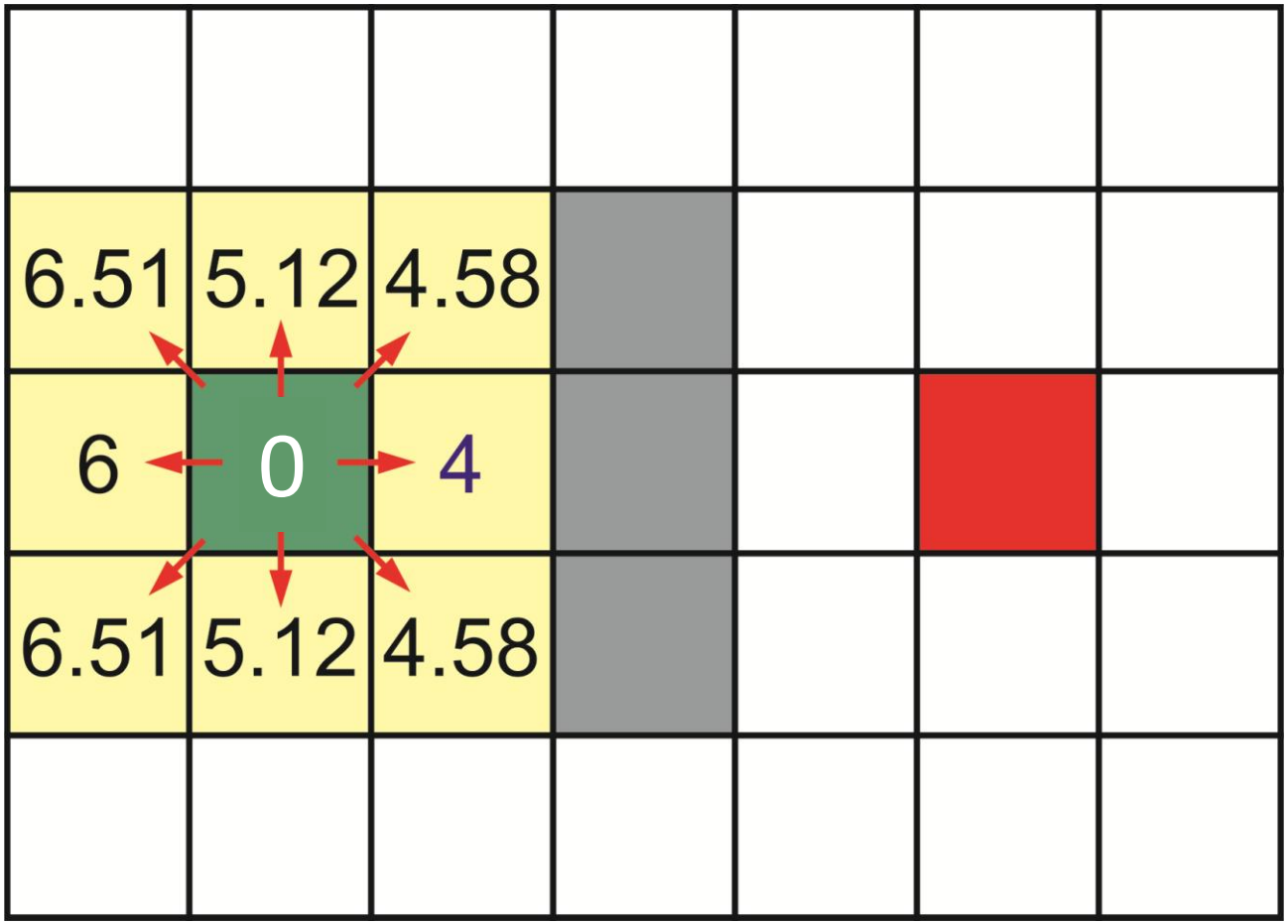
- Calcula-se então todos os custos F para todas as células vizinhas (conectividade 8) da célula atual.



$$F = g + h$$

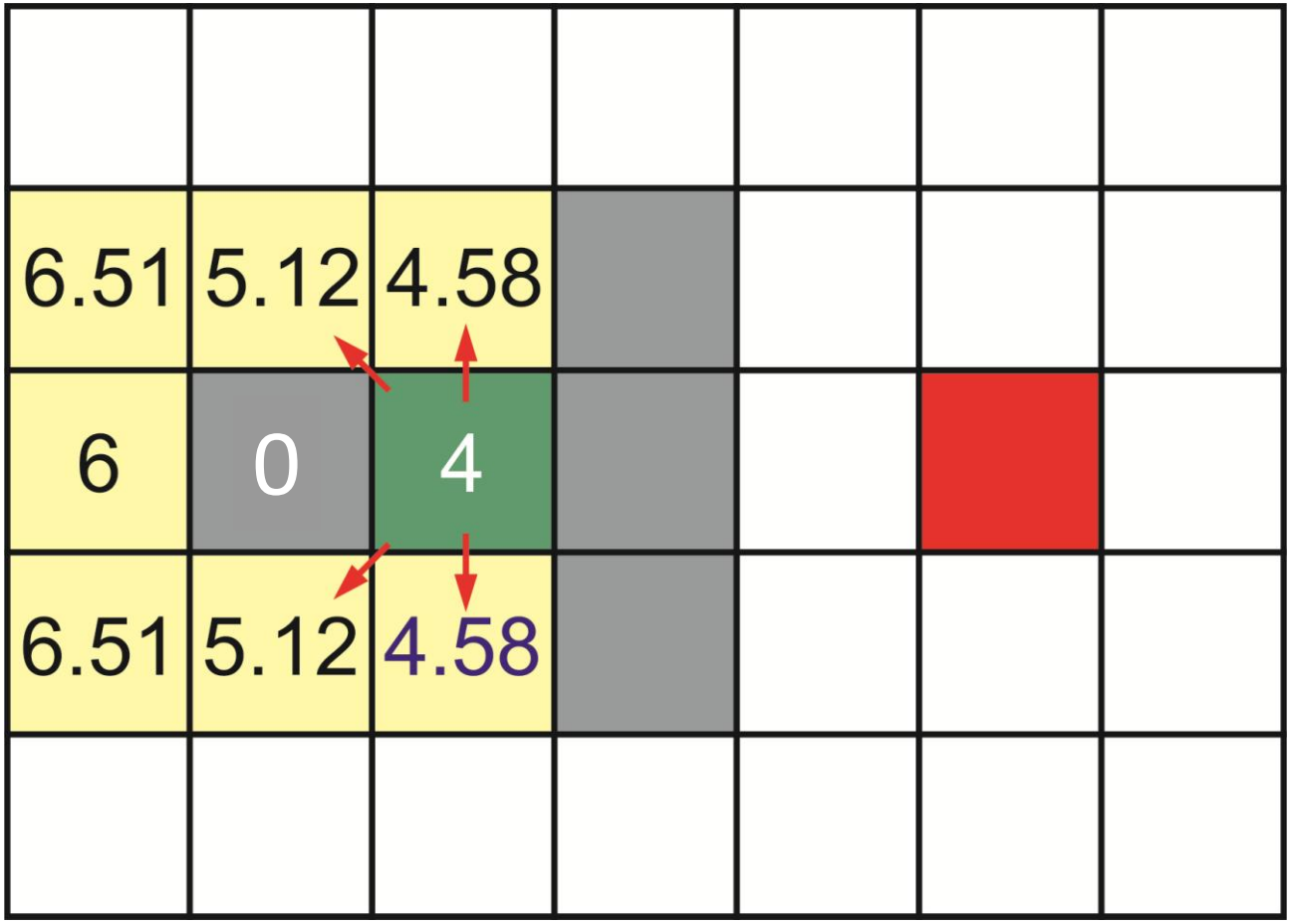
- A **célula vizinha de menor custo será a escolhida para ser a atual**, e a célula atual será incluída na lista de **Fechadas**.
- Uma célula atual $\mathbf{C_{ij}}$ que é incluída nas listas sempre possui anotada a célula $\mathbf{A_{ij}}$ pela qual se chegou em $\mathbf{C_{ij}}$.
- A célula inicial tem um custo de 0

A lista de células abertas está em amarelo e a de células fechadas em cinza.
A célula verde é a célula atual e a vermelha é a célula objetivo.



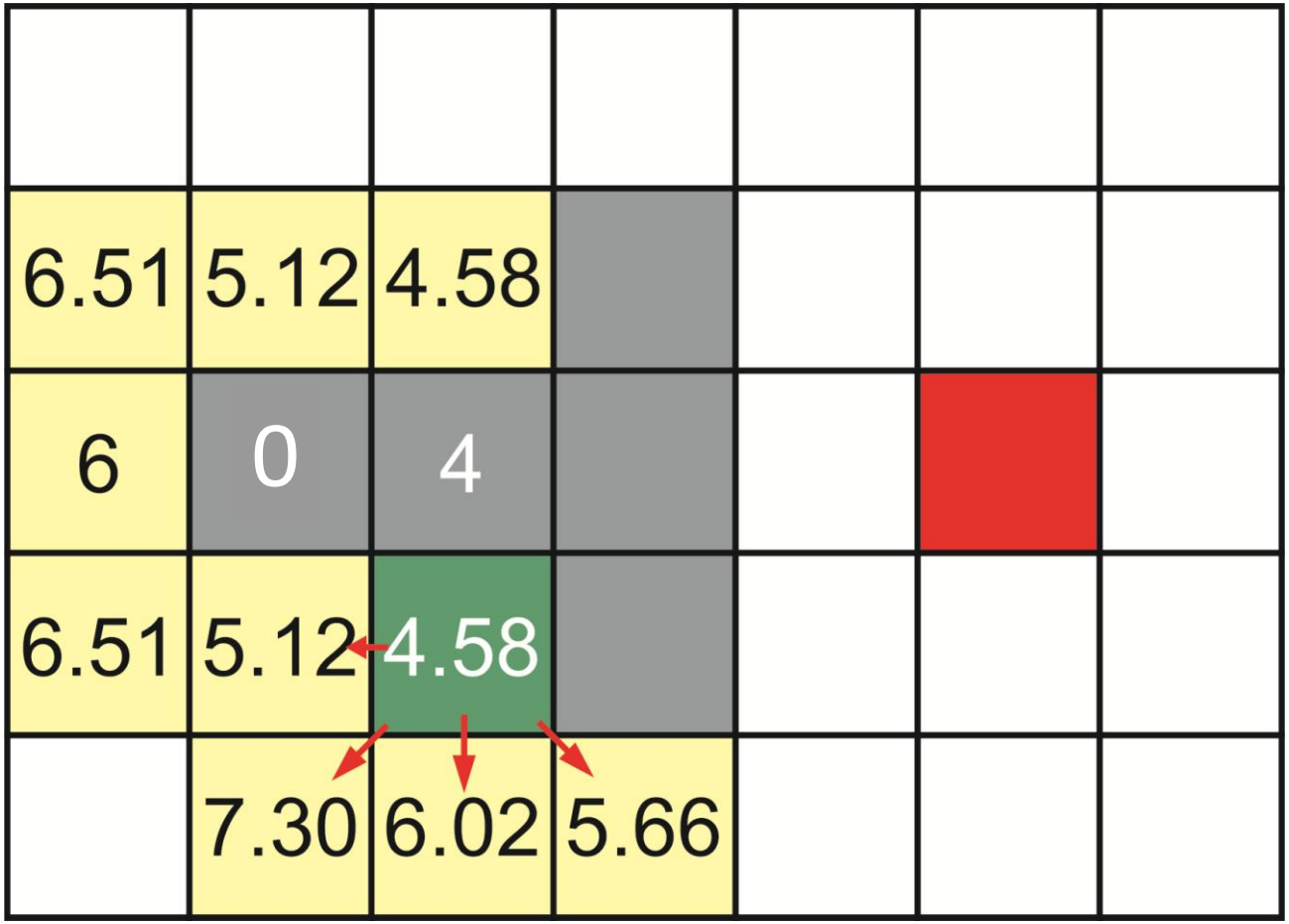
- Repete-se a análise com o custo **F** das células vizinhas à nova célula atual, expandindo a lista de células **Abertas**.
- Exclui-se da análise de cálculo do custo **F** todas as células vizinhas que estão contidas na lista de células fechadas;

A lista de células abertas está em amarelo e a de células fechadas em cinza.
A célula verde é a célula atual e a vermelha é a célula objetivo.



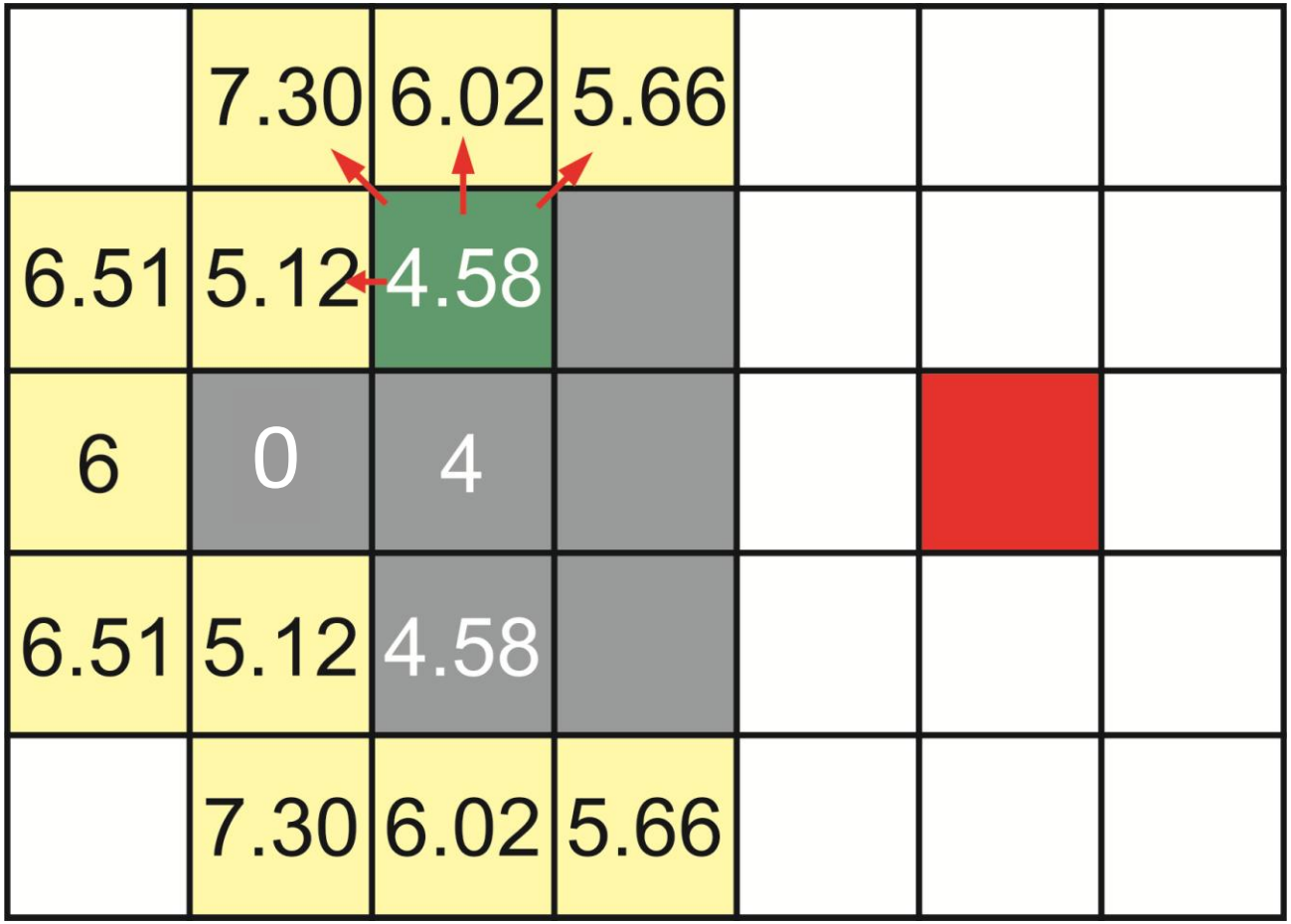
- Repete-se a análise com o custo **F** das células vizinhas à nova célula atual, expandindo a lista de células **Abertas**.
- Exclui-se da análise de cálculo do custo **F** todas as células vizinhas que estão contidas na lista de células fechadas.

A lista de células abertas está em amarelo e a de células fechadas em cinza.
A célula verde é a célula atual e a vermelha é a célula objetivo.



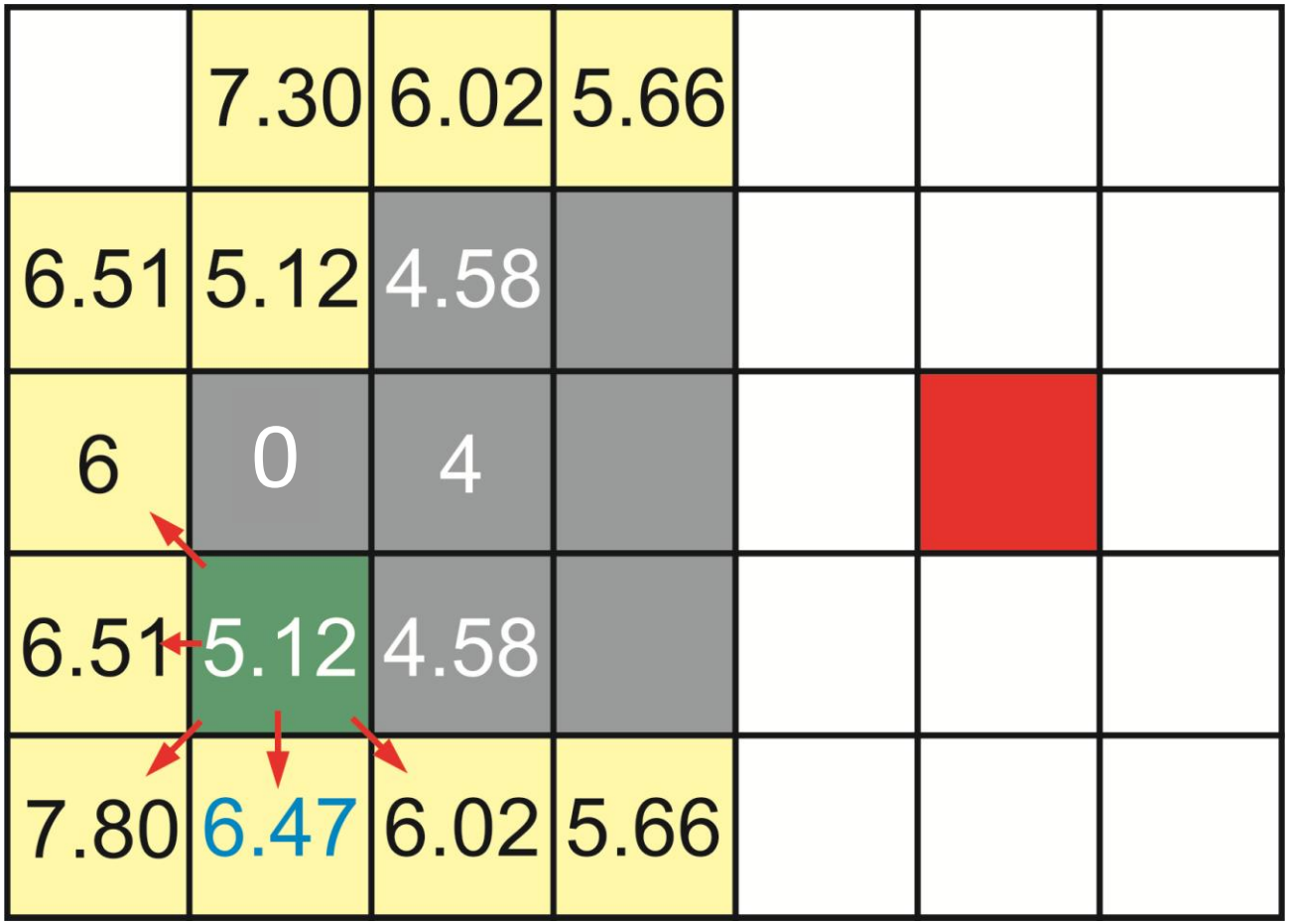
- Repete-se a análise com o custo **F** das células vizinhas à nova célula atual, expandindo a lista de células **Abertas**.
- Exclui-se da análise de cálculo do custo **F** todas as células vizinhas que estão contidas na lista de células fechadas.

A lista de células abertas está em amarelo e a de células fechadas em cinza.
A célula verde é a célula atual e a vermelha é a célula objetivo.



- É possível que um custo **F** menor/maior que o já armazenado para a célula seja encontrado quando se chega na célula por outros caminhos diferentes.
- Se menor, substitui-se **F**, senão mantém-se o antigo.

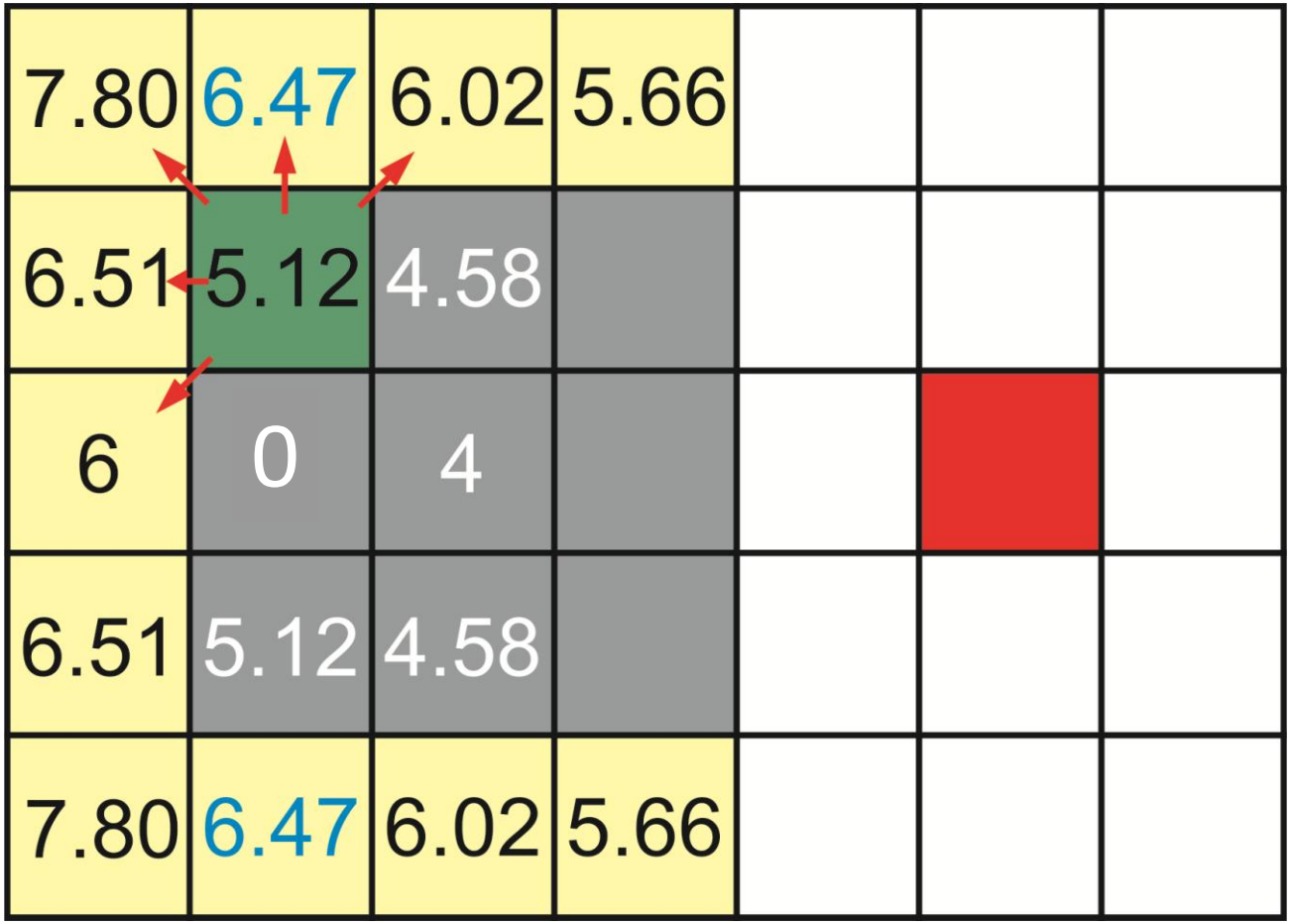
A lista de células abertas está em amarelo e a de células fechadas em cinza.
A célula verde é a célula atual e a vermelha é a célula objetivo.



Quando um custo F menor é encontrado, quer dizer que existe uma maneira menos custosa de chegar naquela célula (por outra célula). Assim, substitui-se o custo F e também a célula que deu origem à célula atual.

- É possível que um custo **F** menor/maior que o já armazenado para a célula seja encontrado quando se chega na célula por outros caminhos diferentes.
- Se menor, substitui-se **F**, senão mantém-se o antigo.

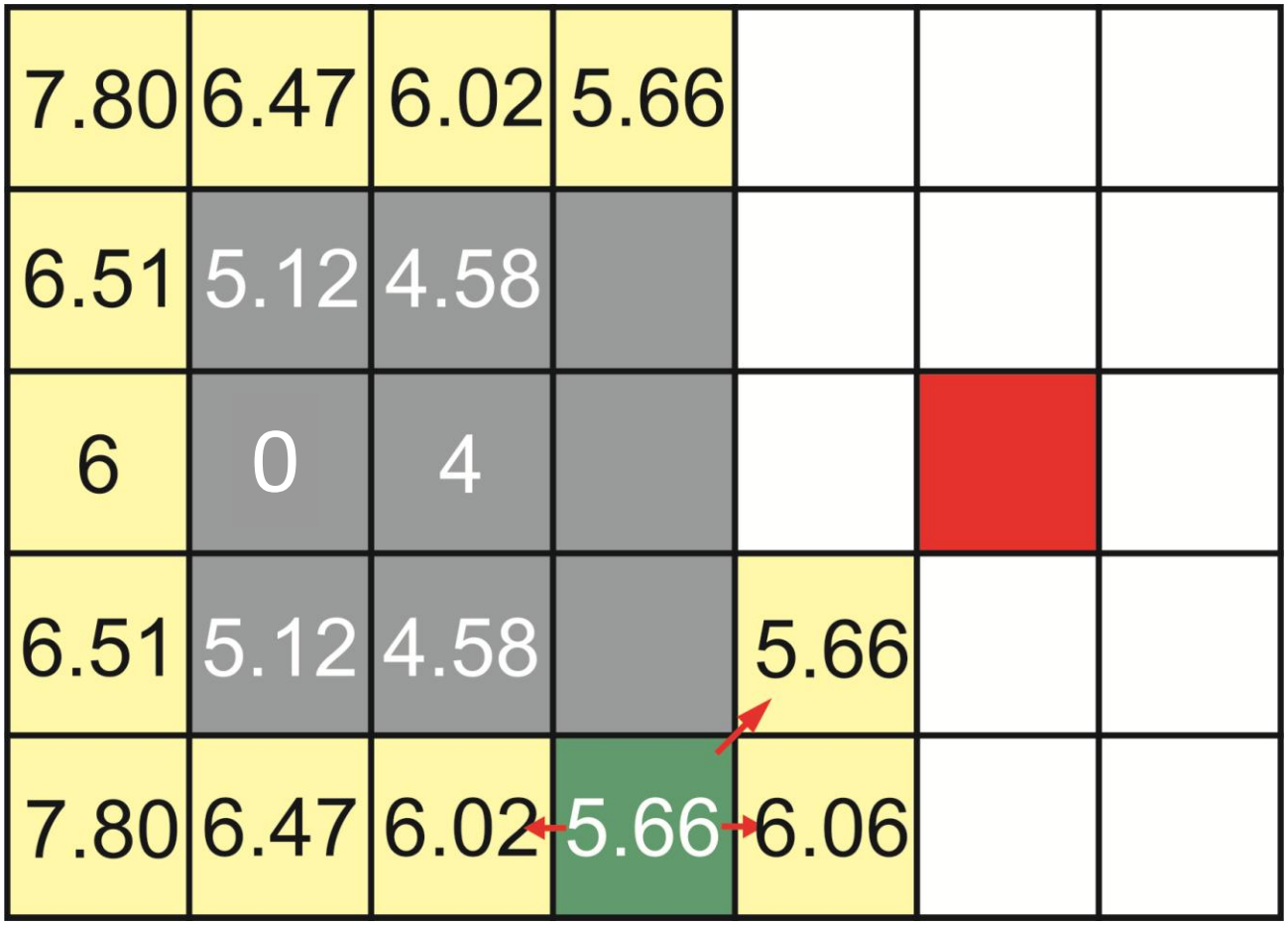
A lista de células abertas está em amarelo e a de células fechadas em cinza.
A célula verde é a célula atual e a vermelha é a célula objetivo.



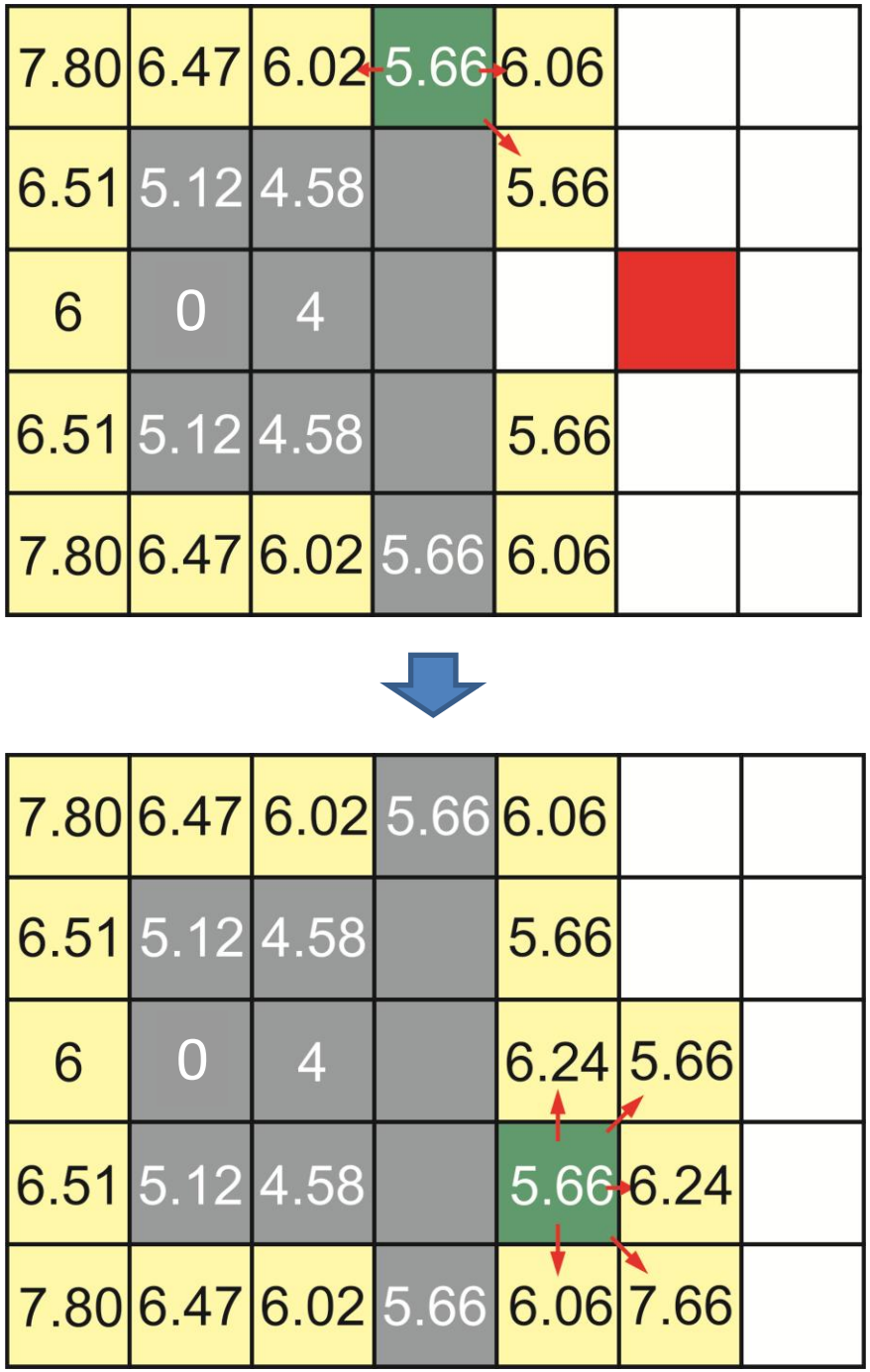
Quando um custo F menor é encontrado, quer dizer que existe uma maneira menos custosa de chegar naquela célula (por outra célula). Assim, substitui-se o custo F e também a célula que deu origem à célula atual.

- Segue-se com o algoritmo, sempre escolhendo como próxima célula a que produz o menor custo (*best-first*).
- Se mais de uma célula possuir o melhor valor, escolhe-se aquela que foi colocada em primeiro lugar nas listas.
- O *best-first* diminui o tempo computacional da solução.

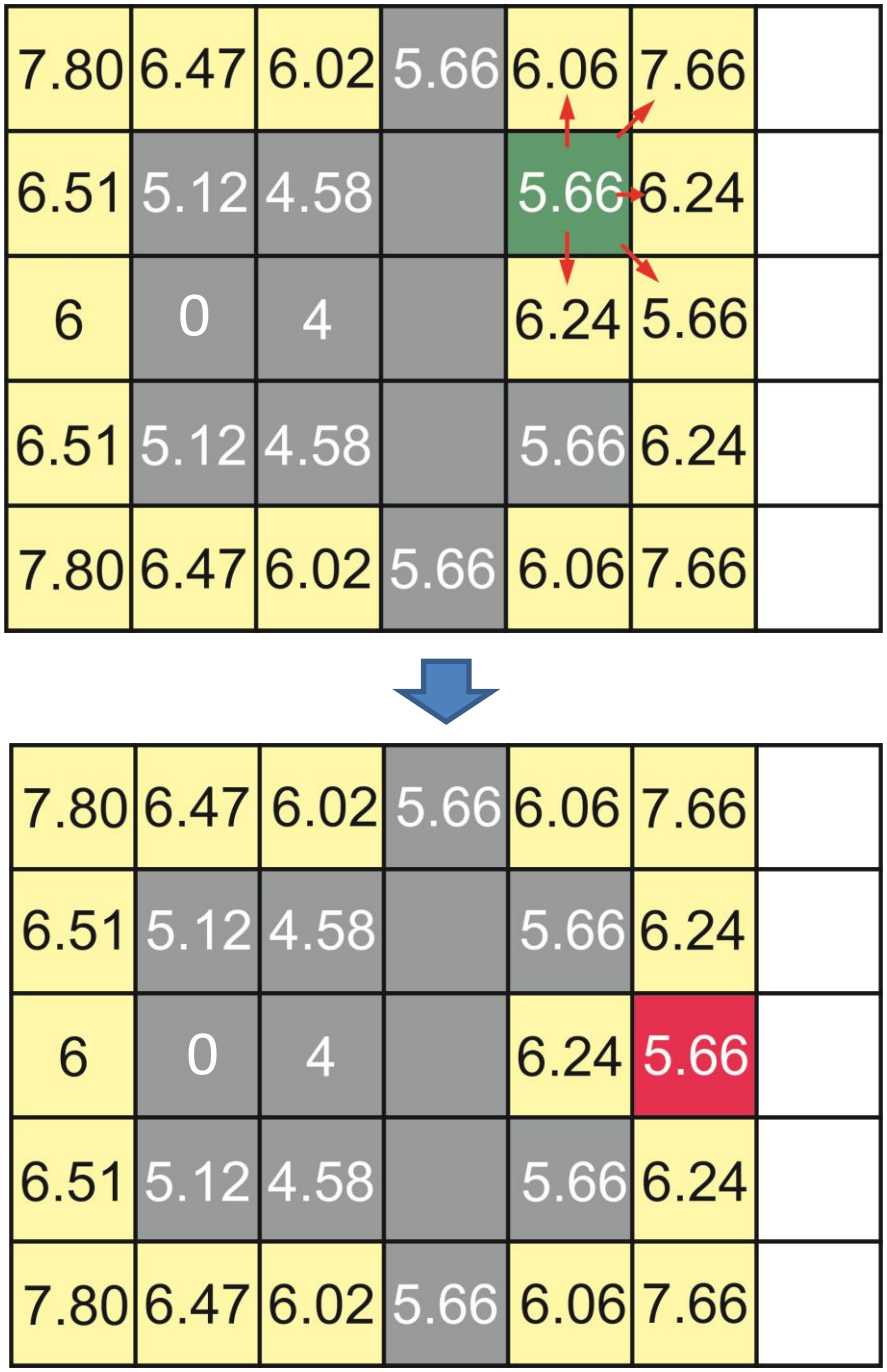
A lista de células abertas está em amarelo e a de células fechadas em cinza.
A célula verde é a célula atual e a vermelha é a célula objetivo.



Continuação:



Continuação:

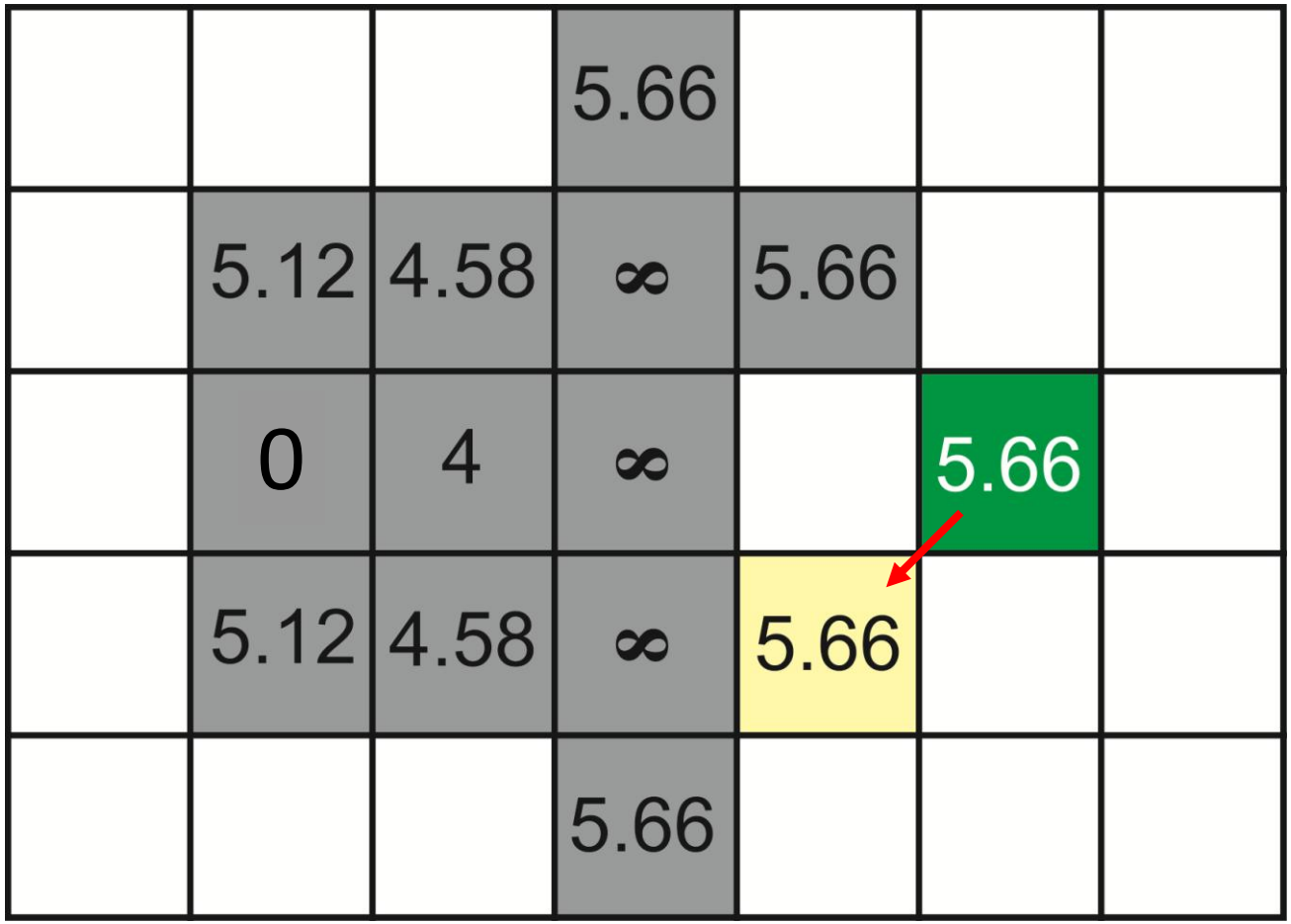


Final:
O algoritmo termina quando a célula atual é a célula objetivo.

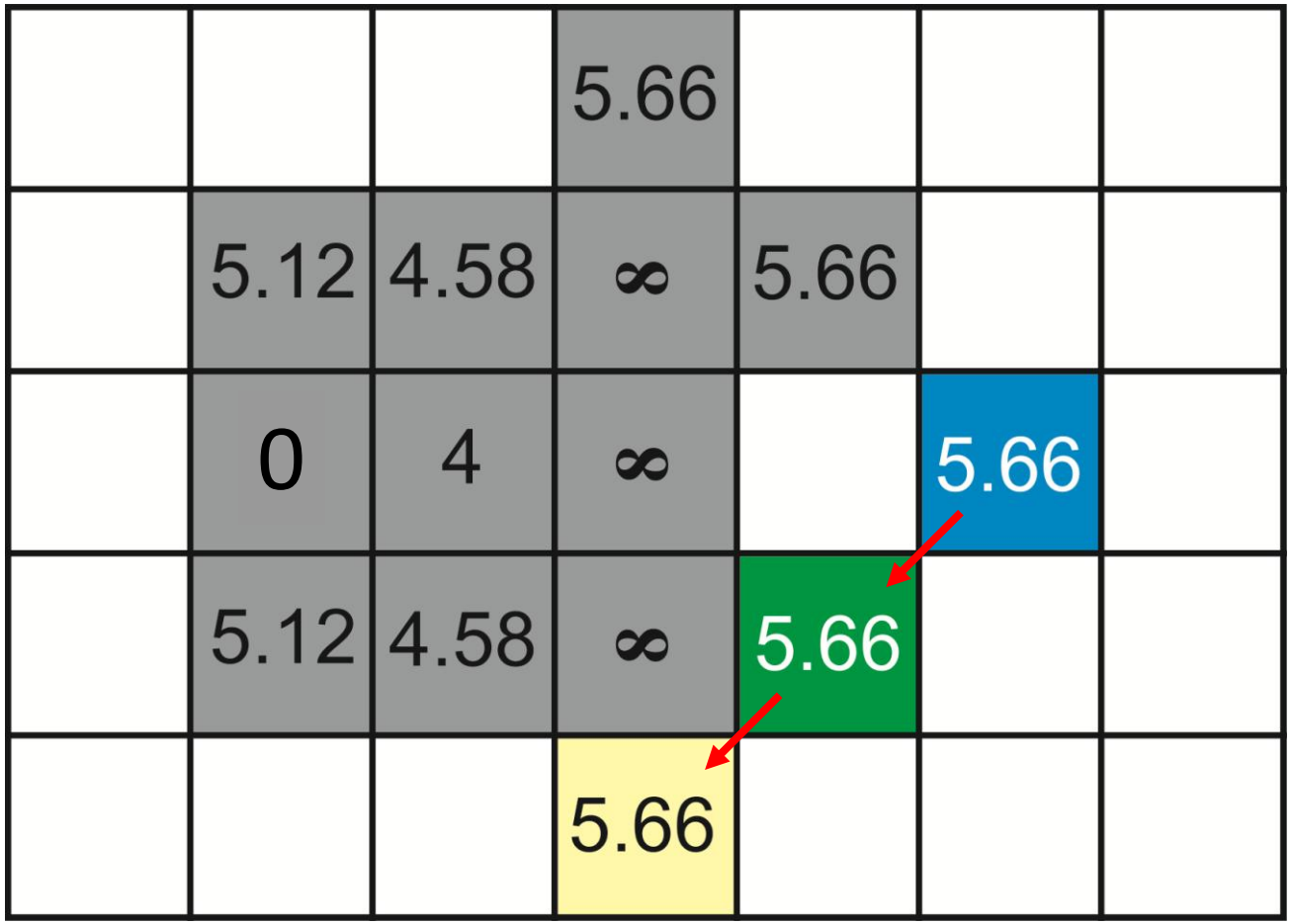
- A primeira fase do algoritmo termina quando a célula atual é a própria célula objetivo.
- Podemos observar que a lista de células Abertas (amarela) e Fechadas (cinza), expandidas pelo algoritmo.

7.80	6.47	6.02	5.66	6.06	7.66	
6.51	5.12	4.58		5.66	6.24	
6	0	4		6.24	5.66	
6.51	5.12	4.58		5.66	6.24	
7.80	6.47	6.02	5.66	6.06	7.66	

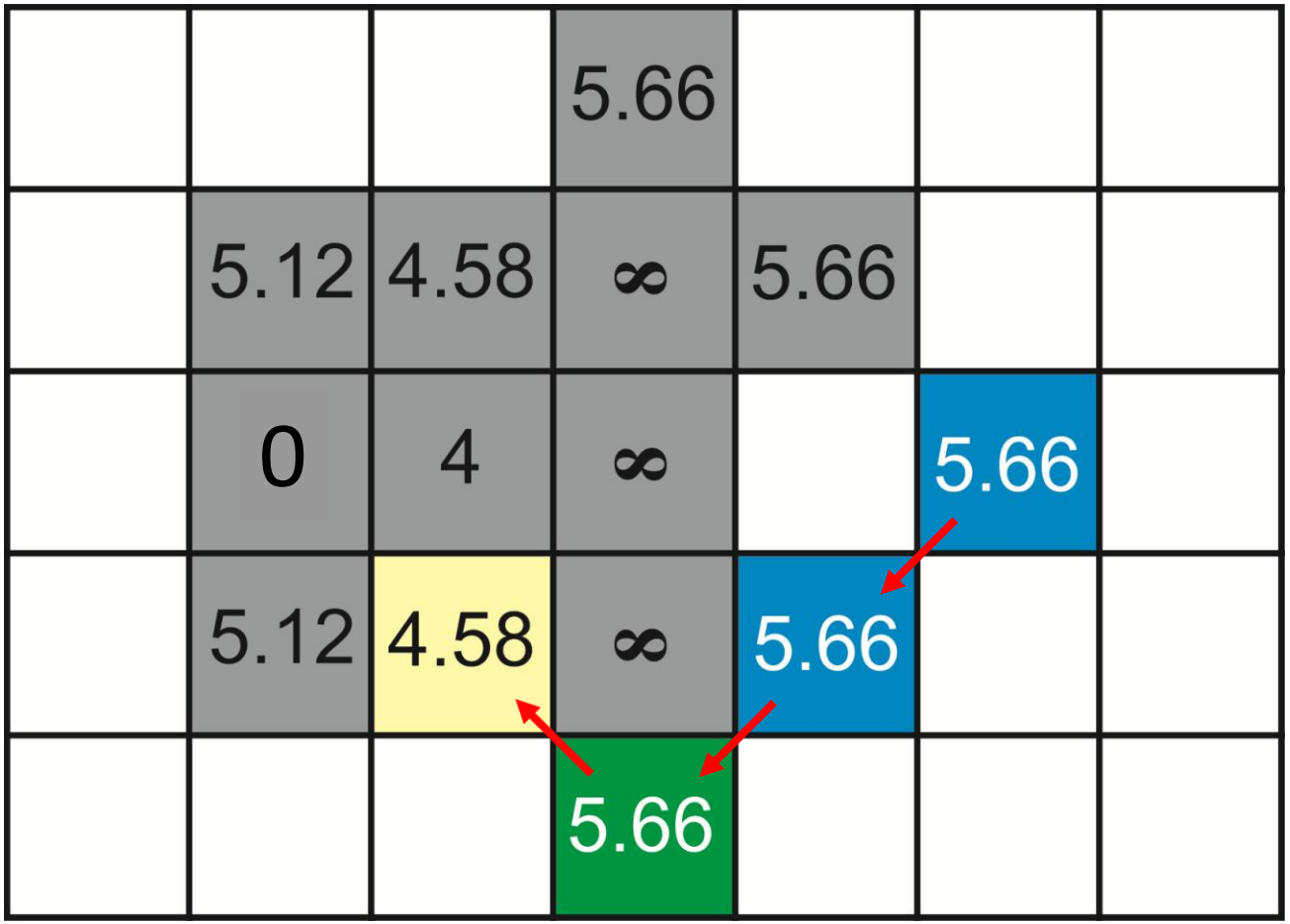
- Quando se chega à célula objetivo, encerra-se a análise **F**.
- Agora, da lista de células **Fechadas**, anda-se de trás para frente (da célula final até a célula inicial) escolhendo as células vizinhas que levaram ao até a célula atual.



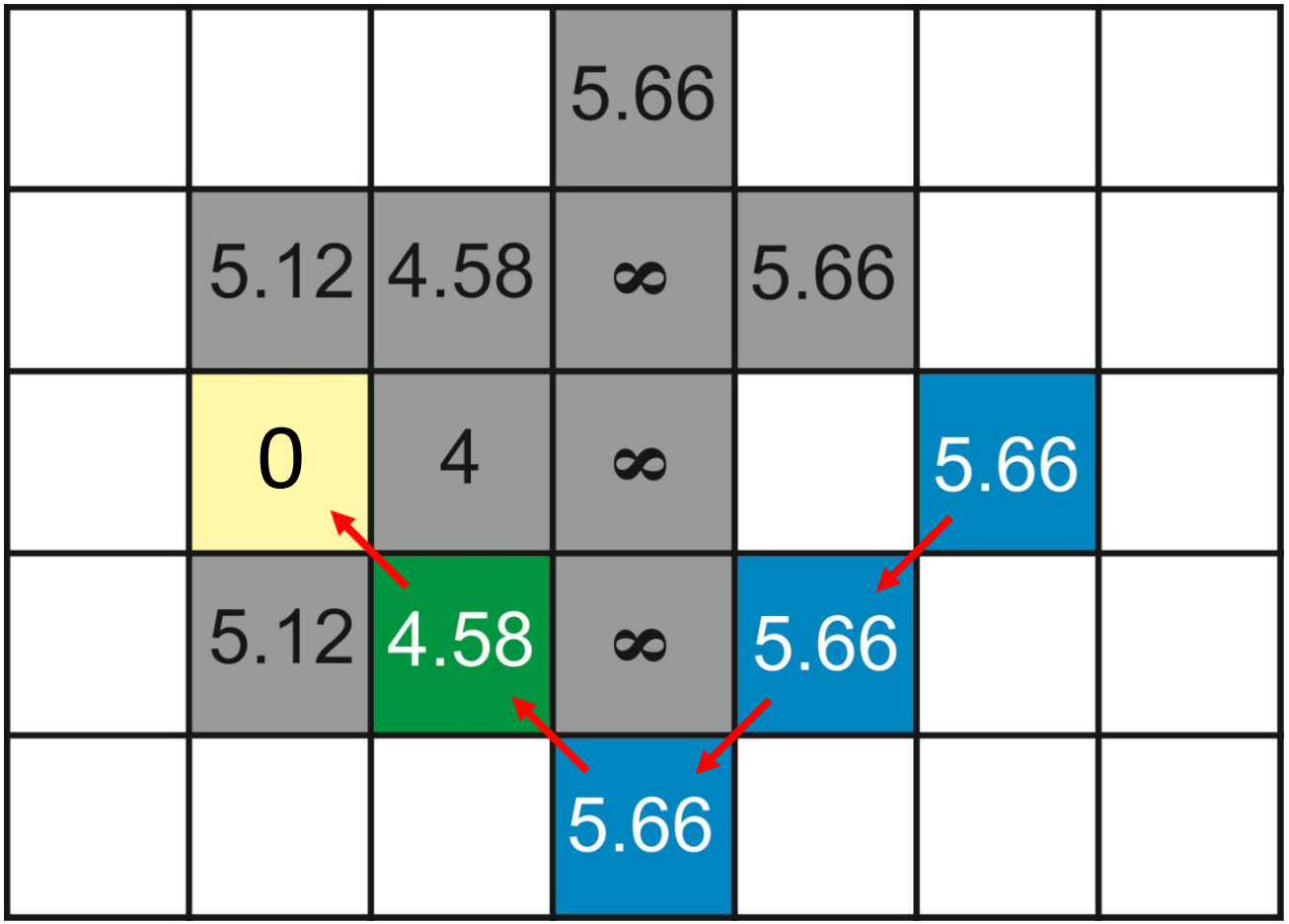
- Quando se chega à célula objetivo, encerra-se a análise **F**.
- Agora, da lista de células fechadas, anda-se de trás para frente (da célula final até a célula inicial) escolhendo as células vizinhas que levaram ao até a célula atual.



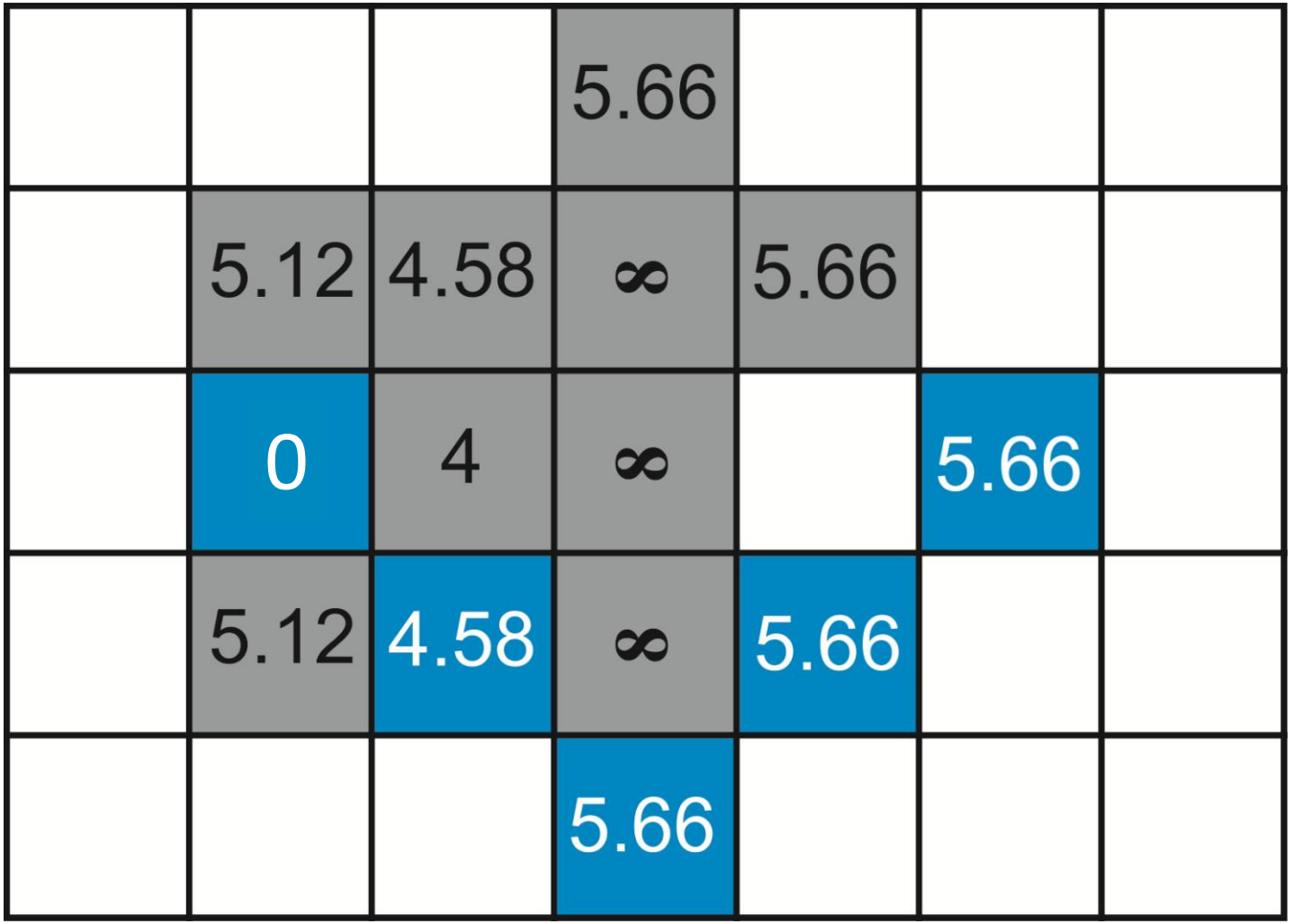
- Retrocedendo de trás para frente, sempre analisando de qual célula vizinha a célula atual veio, o caminho ótimo vai se desenhando.



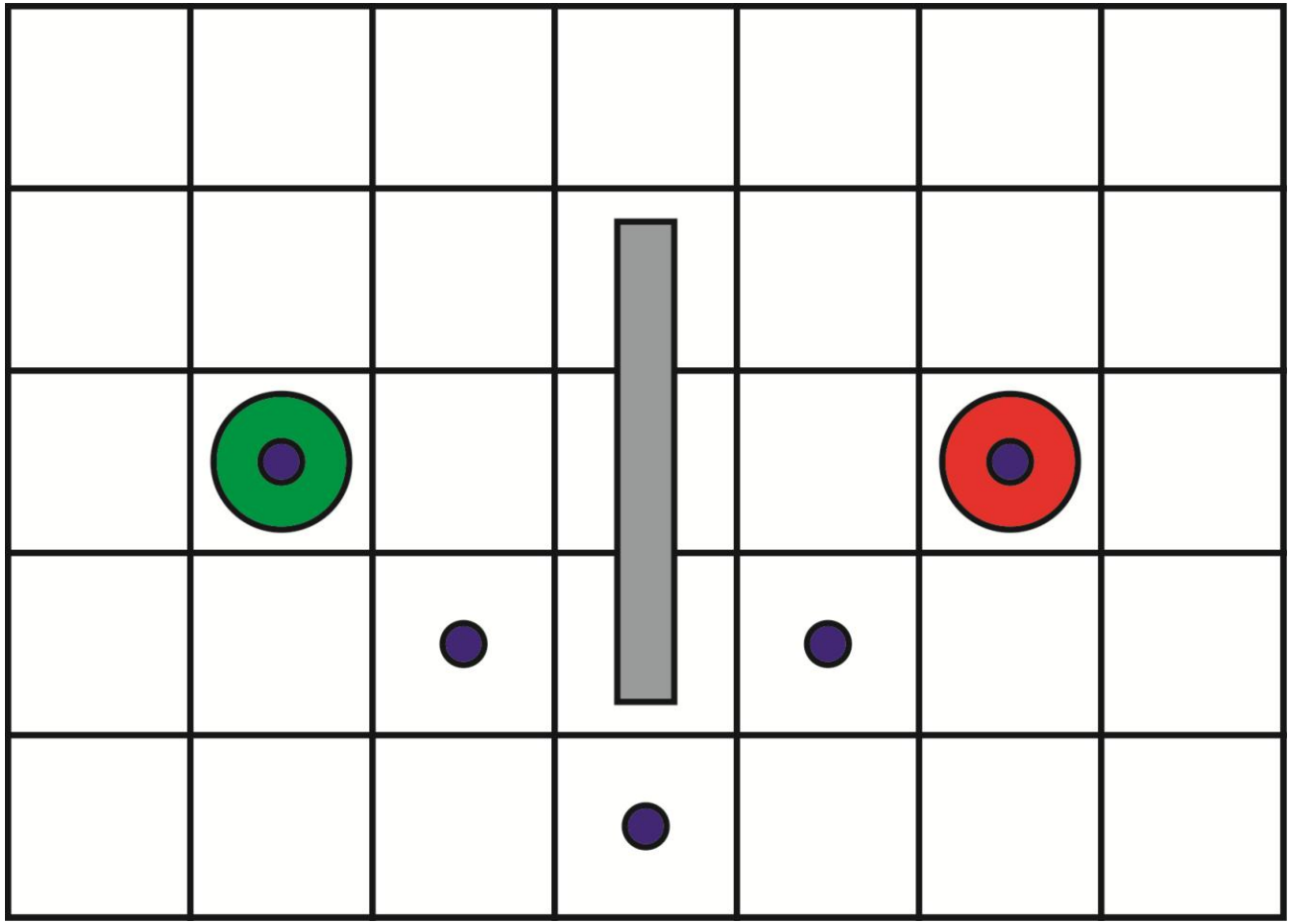
- Retrocedendo de trás para frente, sempre analisando de qual célula vizinha a célula atual veio, o caminho ótimo vai se desenhando.



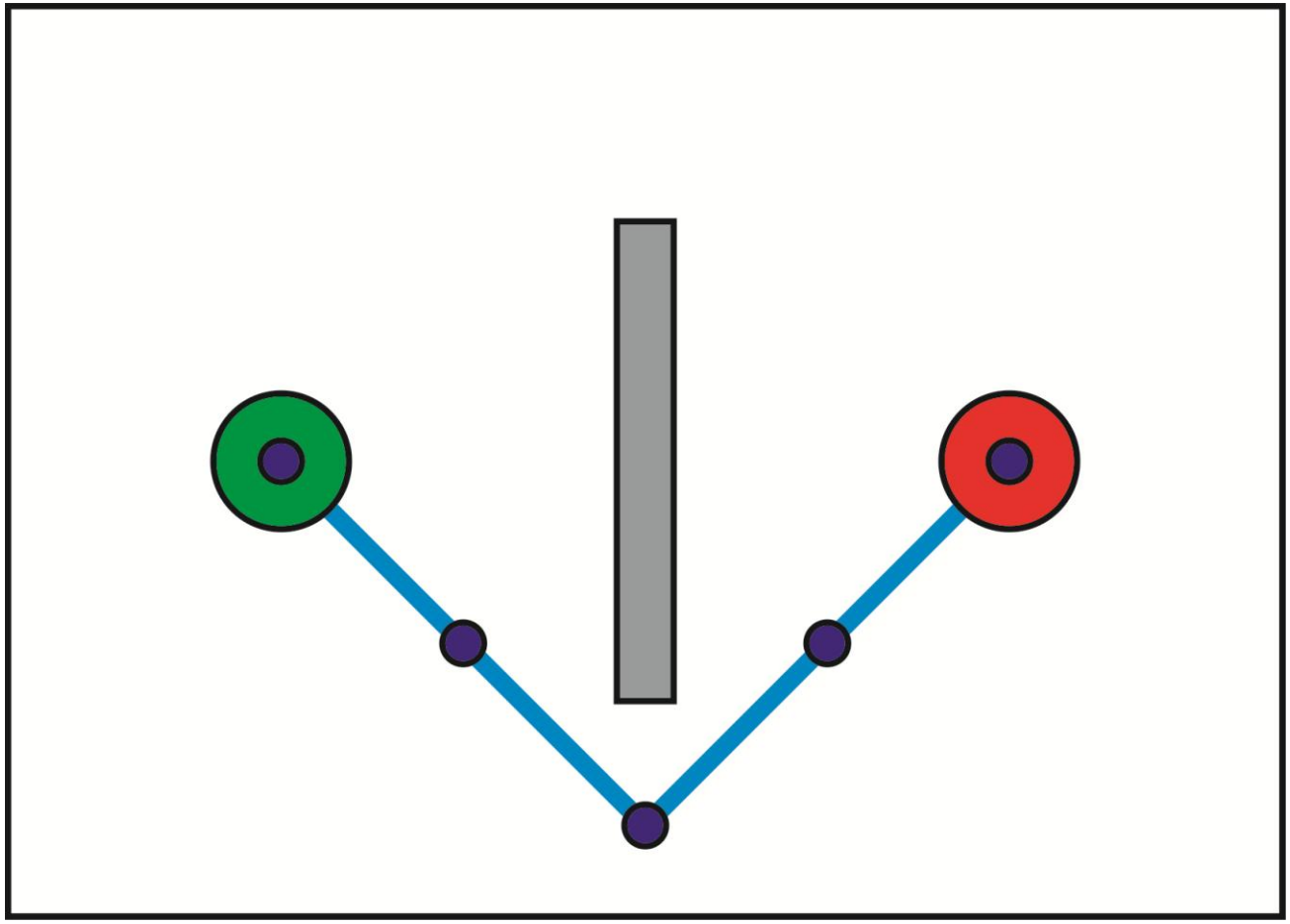
- Após retroceder até a célula inicial, o algoritmo se encerra e o caminho ótimo, de acordo com a conectividade escolhida, é encontrado.



- Assim, é possível retroceder ao mapa original (contínuo) e ofertar ao robô uma sequência de pontos a serem seguidos de forma que estes pontos formam o caminho mais curto (ótimo) do ponto inicial ao ponto objetivo.



- Assim, é possível retroceder ao mapa original, contínuo, e ofertar ao robô uma sequência de pontos a serem seguidos de forma que estes pontos formam o caminho mais curto (ótimo) do ponto inicial ao ponto objetivo.

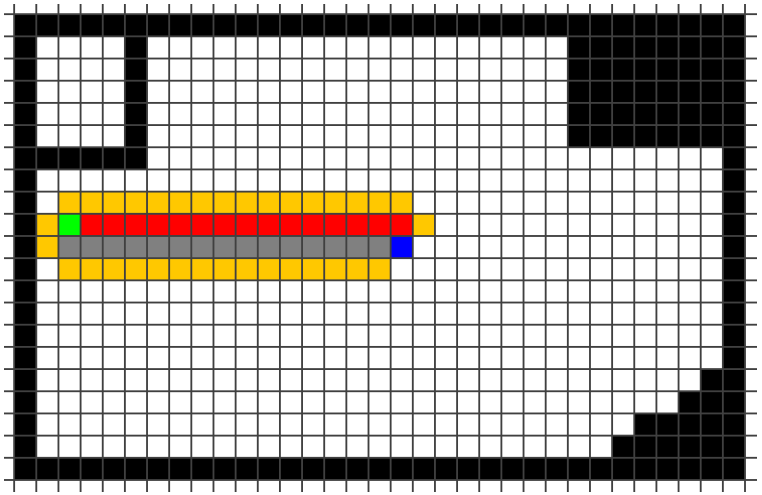


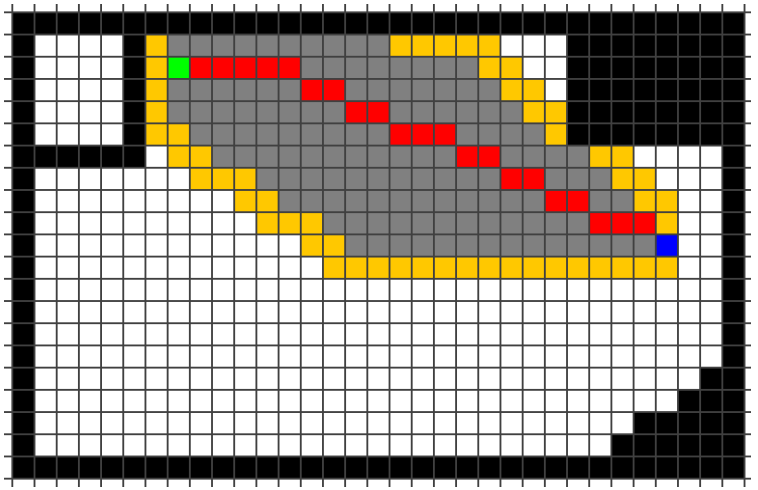
Comentários

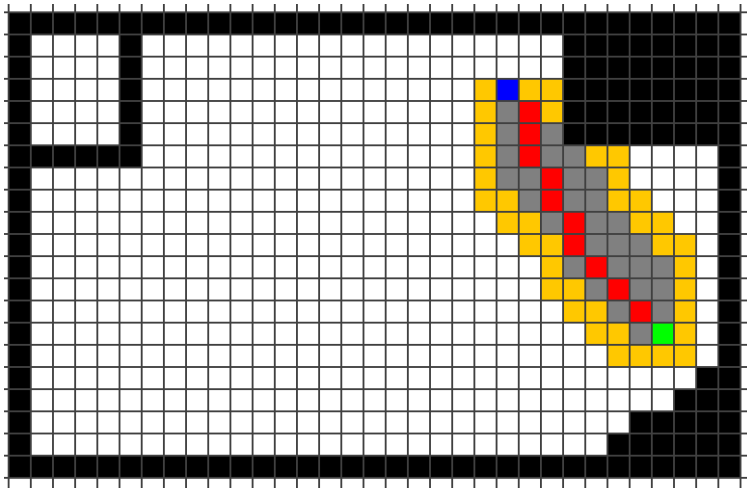
- **Completude probabilística** é uma propriedade que garante a convergência de um *pathfinder* para uma solução do mapa quando o tempo de tender ao infinito.
- O algoritmo A* possui **completude**, que é uma propriedade mais forte que a completude probabilística. A completude garante que a resposta, se existir, será encontrada em tempo computacional hábil. Caso não existir solução, o algoritmo retorna "**falha"**.
- Uma heurística **admissível** (otimista) é aquela que sempre subestima a distância verdadeira do ponto atual até o final.
- O algoritmo terá **solução ótima** se tiver heurística admissível.
- Se a heurística for **inadmissível**, a solução ótima não é garantida, podendo resultar em um caminho **maior** que o mínimo existente.
- Mesmo com uma heurística inadmissível, **sempre** haverá um caminho encontrado se existir solução para o mapa.

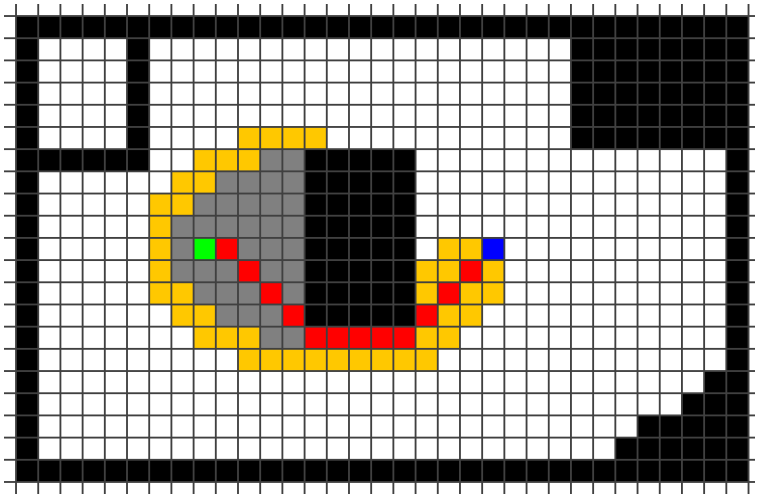
Exemplos

- Exemplos do algoritmo A* no ambiente da disciplina: **caminho em vermelho**, **fronteira em amarelo** e **cinza as células na *closed list***.









Pseudocódigo: Fase de Ida

```
function A*(start,goal)
     closedset := the empty set
     openset := {start}
     came_from := the empty map
     g_score[start] := 0
     f_score[start] := 0
     while openset is not empty
         current := the node in openset having the lowest f_score[] value
         if current = goal
             return reconstruct_path(came_from, goal)
         remove current from openset
         add current to closedset
         for each neighbor in neighbor_nodes(current)
             if neighbor in closedset
                 continue
             tentative_g_score := g_score[current] +
   dist_between(current,neighbor)

             if neighbor not in openset or tentative_g_score <=
   g_score[neighbor]
                 came_from[neighbor] := current
                 g_score[neighbor] := tentative_g_score
                 f_score[neighbor] := g_score[neighbor] +
   heuristic_cost_estimate(neighbor, goal)
                 if neighbor not in openset
                     add neighbor to openset
     return failure
```

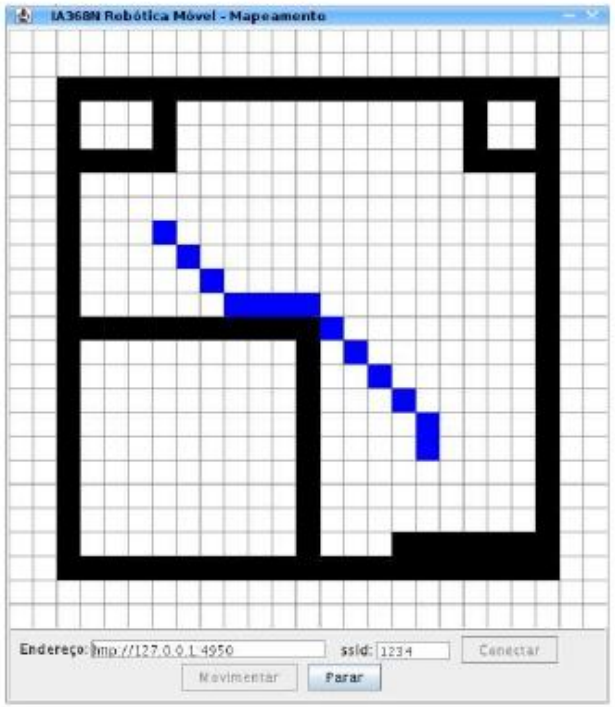
Pseudocódigo: Fase de Volta

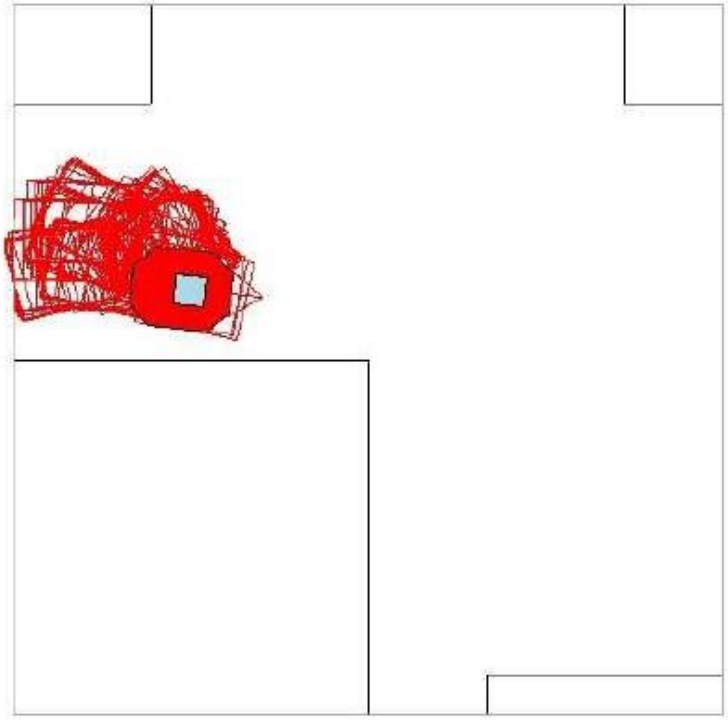
```
function reconstruct_path(came_from, current_node)
     if came_from[current_node] is set
         p := reconstruct_path(came_from, came_from[current_node])
         return (p + current_node)
     else
         return current_node
```

*****Pseudocódigo obtido de:**
http://en.wikipedia.org/wiki/A*_search_algorithm

A* custo do terreno e robô

- Custo do terreno (das células) pode variar:
  - lama, pântano, terra batida, tapetes etc.
- Caminhos suaves:
  - A* pathfinding acha o caminho mais curto, mas não necessariamente o mais suave (conectividade!!!)
  - Há problemas com colisão: passa perto de obstáculos.





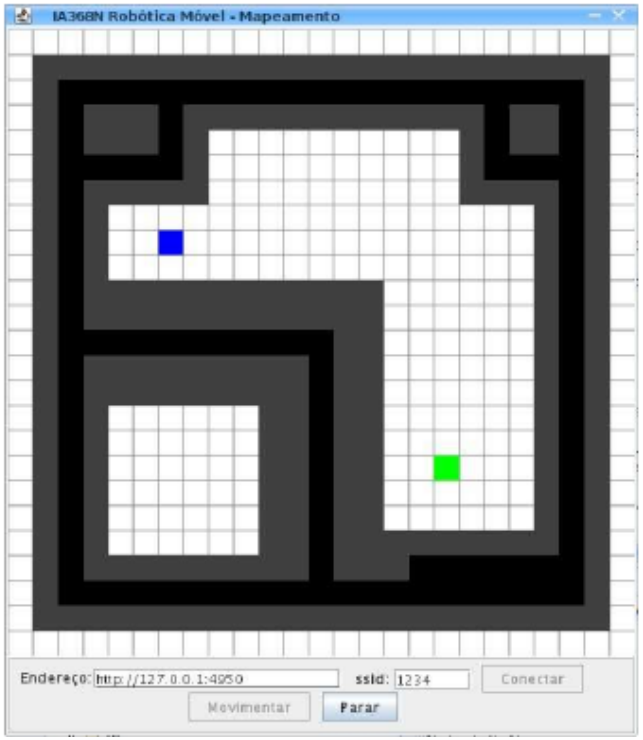
A* custo do terreno e robô

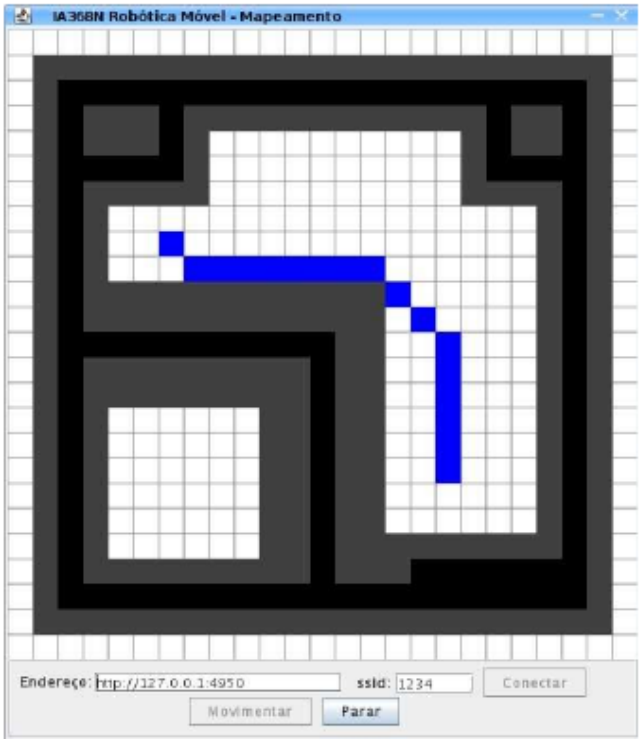
- Tática: utilizar custos variados para as células.
  - Quanto mais próxima a célula estiver do obstáculo, maior o seu custo, forçando o algoritmo a escolher células menos custosas (não tão próximas aos obstáculos).

Células pretas: custo ∞

Células em branco: livres com custo 1.

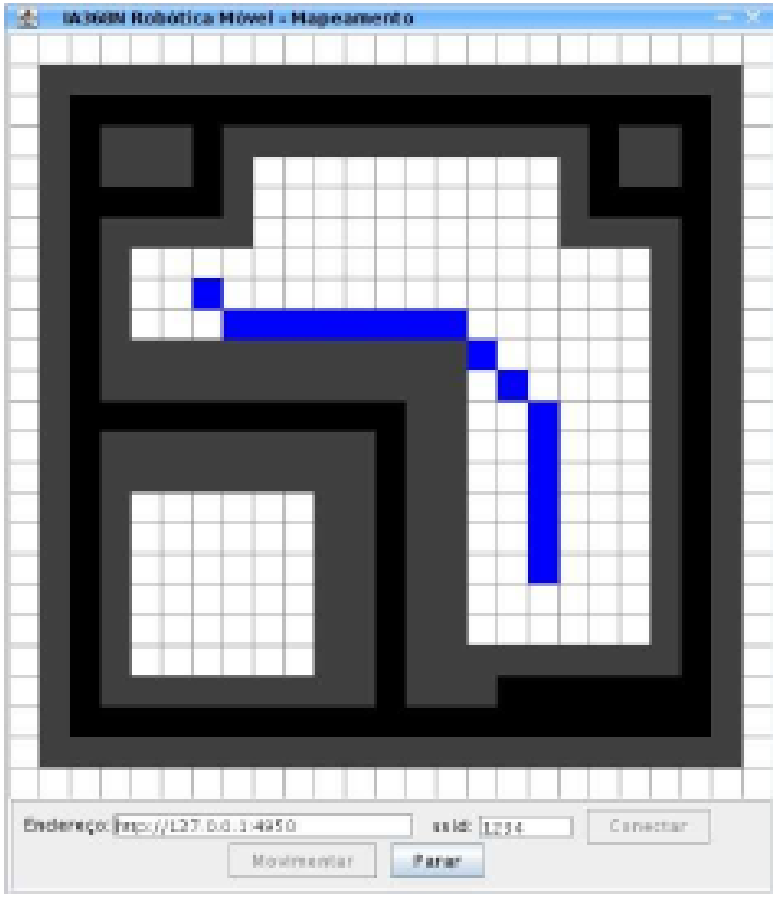
Células em cinza: livres mas com custo maior que 1.

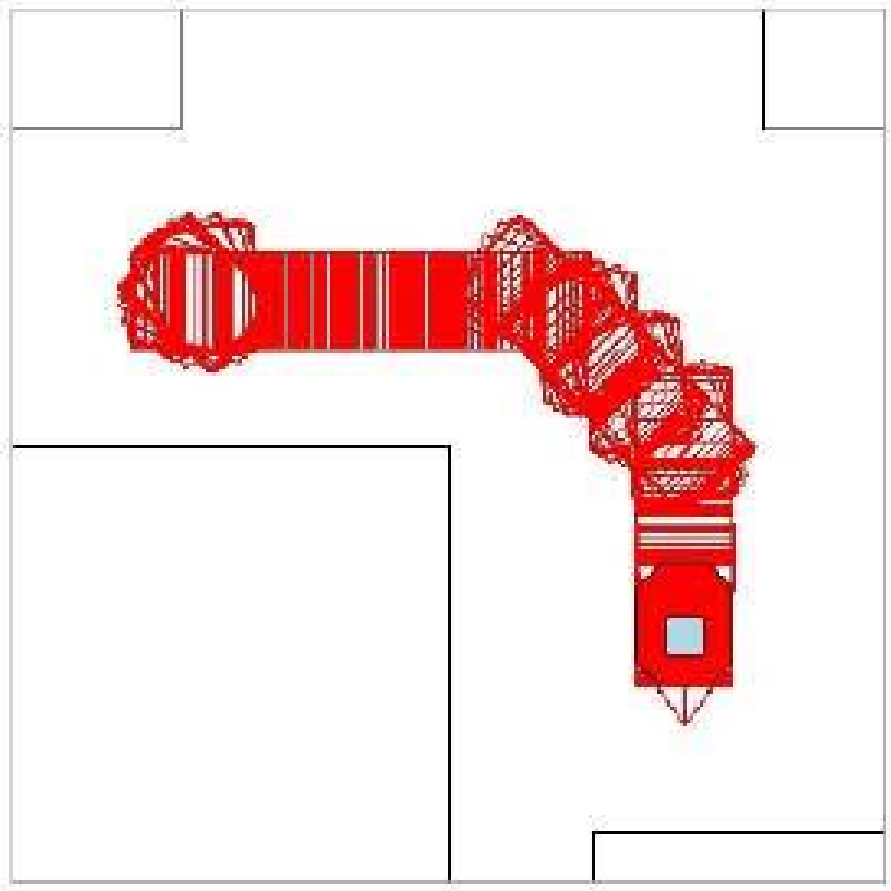




A* custo do terreno e robô

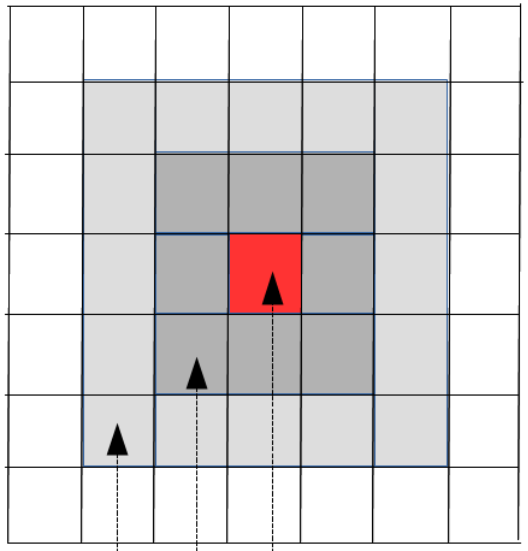
- Resultado: esta tática força o algoritmo a passar longe de obstáculos, aumentando a segurança do robô.





A* custo do terreno e robô

- Outra Tática : Crescimento de obstáculos progressivo



Obstáculo - custo ∞

Adjacentes - custo 10

Adjacentes às adjacentes - custo 3

Demais células com custo 1

